

■ Classical Machine Learning: The Complete Masterclass

Comprehensive Theory, Mathematics, Scratch Code & Scikit-Learn Reference: Lessons 1 through 28

Curriculum Coverage: Linear/Polynomial Regression • Gradient Descent • Regularization (Ridge/Lasso) • Logistic & Softmax • Naive Bayes • KNN • Decision Trees • Random Forests • SVM & SVR • K-Means & DBSCAN • PCA • Pipelines & REST Deployment

Chapter 1: Linear Regression & Optimization from Scratch (Lessons 1–6)

Linear regression models the relationship between dependent target $y \in \mathbb{R}$ and feature vector $x \in \mathbb{R}^d$ as an affine combination: $\hat{y} = w^T x + b = X w$.

1. Closed-Form Normal Equation vs. Gradient Descent:

- **Ordinary Least Squares (OLS) Loss:** $J(w) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2m} \|Xw - y\|^2$.
- **Normal Equation:** Setting $\nabla J(w) = 0$ implies $w^* = (X^T X)^{-1} X^T y$ ($O(d^3)$ complexity, exact analytic solution).
- **Batch Gradient Descent Update:** $w := w - \eta \nabla J(w) = w - \frac{\eta}{m} X^T (Xw - y)$ ($O(m \cdot d)$ per step).

```
# Lesson 3 & 4: Linear Regression from Scratch (Normal Equation & Gradient Descent)
import numpy as np
```

```
class LinearRegressionScratch:
    def __init__(self, lr=0.01, n_iters=1000, method='normal_equation'):
        self.lr, self.n_iters, self.method = lr, n_iters, method
        self.weights, self.bias = None, None

    def fit(self, X, y):
        m, d = X.shape
        if self.method == 'normal_equation':
            X_b = np.c_[np.ones((m, 1)), X] # Add bias intercept column
            theta_best = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y
            self.bias, self.weights = theta_best[0], theta_best[1:]
        else: # Gradient Descent
            self.weights, self.bias = np.zeros(d), 0.0
            for _ in range(self.n_iters):
                y_pred = X @ self.weights + self.bias
                error = y_pred - y
                self.weights -= self.lr * (1/m) * (X.T @ error)
                self.bias -= self.lr * (1/m) * np.sum(error)

    def predict(self, X):
        return X @ self.weights + self.bias
```

Chapter 2: Regularization: Ridge (L2), Lasso (L1) & ElasticNet (Lessons 7–8)

Regularization penalizes large weight magnitudes to mitigate multicollinearity and overfitting (variance reduction):

REGULARIZATION OBJECTIVE FORMULATIONS

1. Ridge (L2 / Tikhonov): $J(w) = \text{MSE}(w) + \alpha \cdot \sum(w_j^2) \implies w = (X^T X + \alpha I)^{-1} X^T y$
 - Shrinks coefficients smoothly toward zero; never sets them to exact 0.
2. Lasso (L1 / Sparsity): $J(w) = \text{MSE}(w) + \alpha \cdot \sum(|w_j|) \implies$ Diamond constraint surface
 - Drives uninformative feature weights to EXACT ZERO (automatic feature selection!).
3. ElasticNet (Hybrid): $J(w) = \text{MSE}(w) + r \cdot \alpha \cdot L1 + ((1-r)/2) \cdot \alpha \cdot L2$

Regularization Selection Guide: When to use which? Use **Ridge** as the default when features are dense and correlated. Use **Lasso** when expecting sparse signals with many irrelevant noise features. Use **ElasticNet** when features are highly correlated and $\rho > n$.

Chapter 3: Logistic Regression, Softmax & Evaluation Metrics (Lessons 9–12)

Logistic regression maps linear projections to calibrated probabilities via the Sigmoid activation $\sigma(z) = \frac{1}{1 + e^{-z}}$. Training minimizes the Log-Loss (Binary Cross-Entropy):

```
# Lessons 9 & 10: Binary Logistic Regression from Scratch
class LogisticRegressionScratch:
    def __init__(self, lr=0.01, n_iters=1000):
        self.lr, self.n_iters = lr, n_iters
        self.weights, self.bias = None, None

    def _sigmoid(self, z):
        return 1.0 / (1.0 + np.exp(-np.clip(z, -250, 250)))

    def fit(self, X, y):
        m, d = X.shape
        self.weights, self.bias = np.zeros(d), 0.0
        for _ in range(self.n_iters):
            probs = self._sigmoid(X @ self.weights + self.bias)
            d_w = (1/m) * (X.T @ (probs - y))
            d_b = (1/m) * np.sum(probs - y)
            self.weights -= self.lr * d_w
            self.bias -= self.lr * d_b

    def predict_proba(self, X):
        return self._sigmoid(X @ self.weights + self.bias)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)
```

Comprehensive Classification Diagnostics (Lesson 11):

- **Precision:** $\frac{TP}{TP + FP}$ (Accuracy of positive predictions; crucial when False Positives are costly, e.g. spam filters).
- **Recall (Sensitivity):** $\frac{TP}{TP + FN}$ (Fraction of actual positives detected; crucial for medical diagnoses).
- **F1-Score:** Harmonic mean $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ balancing both metrics.
- **ROC-AUC:** Plots True Positive Rate ($\text{TPR} = \text{Recall}$) vs False Positive Rate ($\text{FPR} = \frac{FP}{TN + FP}$) across all decision thresholds.

Chapter 4: Naive Bayes & K-Nearest Neighbors (KNN) (Lessons 13–16)

Naive Bayes applies Bayes' Theorem under the conditional independence assumption $P(x_1, \dots, x_d | C_k) = \prod_{j=1}^d P(x_j | C_k)$.

K-Nearest Neighbors (KNN) is a non-parametric instance-based learner assigning majority class labels based on distance metrics (L_2 Euclidean / L_1 Manhattan):

```
# Lesson 15: KNN Classifier from Scratch
class KNNClassifierScratch:
    def __init__(self, k=3, metric='euclidean'):
        self.k, self.metric = k, metric

    def fit(self, X, y):
        self.X_train, self.y_train = np.array(X), np.array(y)

    def predict(self, X):
        X = np.array(X)
        return np.array([self._predict_sample(x) for x in X])

    def _predict_sample(self, x):
        if self.metric == 'euclidean':
            distances = np.linalg.norm(self.X_train - x, axis=1)
        else: # Manhattan
            distances = np.sum(np.abs(self.X_train - x), axis=1)
        k_indices = np.argsort(distances)[:self.k]
        k_labels = self.y_train[k_indices]
        return np.bincount(k_labels).argmax()
```

Chapter 5: Decision Trees & Random Forest Ensembles (Lessons 17–20)

Decision Trees perform recursive binary axis-aligned partition search to maximize impurity reduction (Information Gain):

SPLITTING CRITERIA FORMULATIONS

1. Shannon Entropy: $H(S) = - \sum_{k=1}^K p_k \cdot \log_2(p_k)$
2. Gini Impurity (CART): $G(S) = 1 - \sum_{k=1}^K p_k^2$
3. Information Gain: $IG(S, A) = \text{Impurity}(S) - \sum_{v \in A} \left(\frac{|S_v|}{|S|} \right) \cdot \text{Impurity}(S_v)$
4. Regression MSE Split: $\text{Var}(S) = (1/|S|) \cdot \sum_{i \in S} (y_i - \bar{y})^2$

```
# Lesson 20: Random Forest Classifier
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,      # Number of de-correlated decision trees
    max_depth=10,         # Maximum tree depth limit to prevent overfitting
    max_features='sqrt',  # Random feature subsampling at each split (sqrt(d))
    bootstrap=True,       # Sample with replacement
    oob_score=True,       # Out-of-Bag validation evaluation
    random_state=42
)
rf.fit(X_train, y_train)
# Feature Importances based on mean impurity decrease:
importances = rf.feature_importances_
```

Chapter 6: Support Vector Machines & SVR (Lessons 21 & 24)

SVM finds the maximum-margin hyperplane separating classes while tolerating margin violations via slack variables ξ_i (controlled by penalty $C\xi$):

```
# Lessons 21 & 24: Support Vector Classifier & Regressor with Kernels
from sklearn.svm import SVC, SVR

# 1. Non-linear RBF Kernel Classifier:  $K(x, x') = \exp(-\gamma * ||x - x'||^2)$ 
svm_clf = SVC(C=1.0, kernel='rbf', gamma='scale', probability=True)
svm_clf.fit(X_train, y_train)

# 2. Epsilon-Insensitive Support Vector Regressor (SVR)
svr_reg = SVR(C=10.0, epsilon=0.1, kernel='rbf', gamma='scale')
svr_reg.fit(X_train, y_train)
# Predictions inside the +/- epsilon tube incur zero loss!
```

Chapter 7: Clustering (K-Means, DBSCAN) & PCA (Lessons 25–27)

Unsupervised algorithms uncover latent clusters and low-dimensional manifolds without target labels:

```
# Lesson 25: Principal Component Analysis (PCA) from Scratch
class PCA_Scratch:
    def __init__(self, n_components=2):
        self.n_components = n_components
        self.components, self.mean, self.explained_variance_ratio = None, None, None

    def fit(self, X):
        self.mean = np.mean(X, axis=0)
        X_centered = X - self.mean
        # 1. Compute Covariance Matrix:  $\Sigma = (1/m) * X^T X$ 
        cov_matrix = np.cov(X_centered, rowvar=False)
        # 2. Eigenvalue Decomposition:  $\Sigma * v = \lambda * v$ 
        eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
        # 3. Sort eigenvectors by descending eigenvalue magnitude
        idx = np.argsort(eigenvalues)[::-1]
        self.components = eigenvectors[:, idx[:self.n_components]]
        self.explained_variance_ratio = eigenvalues[idx[:self.n_components]] / np.sum(eigenvalues)

    def transform(self, X):
        return (X - self.mean) @ self.components
```

```
# Lessons 26 & 27: K-Means & DBSCAN Clustering
from sklearn.cluster import KMeans, DBSCAN

# K-Means: Minimizes Within-Cluster Sum of Squares (Inertia)
kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=42)
cluster_labels = kmeans.fit_predict(X)

# DBSCAN: Density clustering discovering arbitrary shapes without pre-specifying k
dbscan = DBSCAN(eps=0.3, min_samples=5)
db_labels = dbscan.fit_predict(X) # -1 indicates noise outliers!
```

Chapter 8: Scikit-Learn Pipelines & Model Deployment (Lesson 28)

Production ML avoids data leakage by encapsulating feature imputation, categorical encoding, scaling, and estimators into a single callable ‘Pipeline’:

```
# Lesson 28: Production-Grade ColumnTransformer & Pipeline
import joblib
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

num_features = ['age', 'income', 'experience']
cat_features = ['education', 'city']

num_pipeline = Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())])
cat_pipeline = Pipeline([('imputer', SimpleImputer(strategy='most_frequent')), ('ohe', OneHotEncoder(handle_unknown='ignore'))])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_features),
    ('cat', cat_pipeline, cat_features)
])

full_pipeline = Pipeline([('preprocessor', preprocessor), ('regressor', RandomForestRegressor(random_state=42))])
full_pipeline.fit(X_train, y_train)

# Serialize complete end-to-end model for production deployment:
joblib.dump(full_pipeline, 'production_model.pkl')
# In production Flask app: model = joblib.load('production_model.pkl'); pred = model.predict(json_input)
```

Chapter 9: Master Machine Learning Function & API Encyclopedia

Comprehensive reference table covering all critical Scikit-Learn estimators, transformers, metrics, and hyperparameters:

Estimator / Function	Module	Exact Purpose & Critical Hyperparameters
LinearRegression()	sklearn.linear_model	Ordinary Least Squares regression via SVD / Normal Equation. Parameter: fit_intercept.
Ridge(alpha=1.0)	sklearn.linear_model	L2 regularized linear regression. Prevents multicollinearity by shrinking weights smoothly.
Lasso(alpha=1.0)	sklearn.linear_model	L1 regularized regression via coordinate descent. Sets uninformative weights to exact 0.
LogisticRegression()	sklearn.linear_model	Log-Loss classification. Hyperparameters: C (inverse regularization), penalty (l1/l2), multi_class.
KNeighborsClassifier(k)	sklearn.neighbors	Instance-based majority vote classification. Hyperparameters: n_neighbors, metric (minkowski).
DecisionTreeClassifier()	sklearn.tree	Recursive partitioning. Parameters: criterion (gini/entropy), max_depth, ccp_alpha.
RandomForestClassifier()	sklearn.ensemble	Bagging ensemble of de-correlated trees. Parameters: n_estimators, max_features, oob_score.
SVC(C, kernel)	sklearn.svm	Maximum-margin classification with kernels (linear, rbf, poly), slack C, and gamma.
KMeans(n_clusters)	sklearn.cluster	Centroid clustering. Parameters: n_clusters, init='k-means++', n_init. Measures inertia_.
DBSCAN(eps, min_samples)	sklearn.cluster	Density clustering discovering arbitrary non-convex clusters and isolating -1 noise outliers.
PCA(n_components)	sklearn.decomposition	Eigenvalue decomposition of covariance matrix. Measures explained_variance_ratio_.
ColumnTransformer	sklearn.compose	Applies distinct transformation pipelines to categorical and numerical feature subsets in parallel.
Pipeline(steps)	sklearn.pipeline	Chains transformers and estimators sequentially to eliminate data leakage during cross-validation.
GridSearchCV	sklearn.model_selection	Exhaustive hyperparameter tuning with K-fold cross-validation across a specified parameter grid.